

# Claude Code 기반 Windows 안드로이드 앱 개발 환경 구축 — 세션 기록

작성일: 2026-07-01

## 1. 세션 개요

Windows 네이티브 환경(PowerShell)에서 Claude Code를 CLI로 활용해 안드로이드 앱을 개발하기 위한 전체 흐름을 논의하고, 실제로 개발 환경(JDK, Android SDK, 에뮬레이터)을 처음부터 구축한 과정을 기록한다. 사용자는 Claude Code가 이미 설치되어 있고 CLI 중심으로 작업하는 환경이며, 안드로이드 앱 개발은 에뮬레이터를 사용하는 방식으로 진행하기로 했다.

## 2. 전체 개발 플로우 (논의 요약)

### 2.1 개발 환경 준비

- Claude Code 설치 방식: 네이티브 윈도우 설치(PowerShell 기반, Node.js 불필요) 또는 WSL2 경유 설치 중 선택. 본 세션은 네이티브 윈도우 방식으로 진행.
- 네이티브 윈도우에서는 Bash 도구 사용을 위해 Git for Windows 설치가 권장되며, 없으면 PowerShell 이 셸 도구로 대체 사용됨.
- 안드로이드 개발 도구는 Claude Code 와 별개로 직접 설치 필요: JDK 17, Android SDK, 에뮬레이터(AVD) 또는 실기기, Gradle(대부분 프로젝트에 gradlew 래퍼 포함).

### 2.2 프로젝트 시작 및 CLAUDE.md

- Gradle 프로젝트 골격(build.gradle.kts, settings.gradle.kts, app 모듈)이 있는 상태에서 claude 명령으로 세션 시작.
- 프로젝트 루트에 CLAUDE.md 를 작성해 패키지 구조, 빌드 명령, 코딩 컨벤션, SDK 버전 등 컨텍스트를 고정.

### 2.3 개발 루프

- 자연어로 기능 요청 → Claude Code 가 관련 파일 탐색/수정 → ./gradlew build 로 빌드 검증 → 유닛/계측 테스트 실행 → adb install/logcat 으로 실행기·에뮬레이터에서 확인 → Git 커밋.
- 화면 단위/기능 단위로 작게 쪼개어 반복하는 것을 권장.

## 2.4 디버깅 및 배포

- logcat 로그를 Claude Code 에 전달하거나 직접 실행하게 하여 크래시 원인 분석.
- UI 스크린샷 첨부를 통한 시각적 디버깅 가능.
- 서명 키 생성, assembleRelease/bundleRelease(AAB), Play Console 업로드는 수동 또는 별도 CI(GitHub Actions)로 처리.

## 3. 환경 구축 실행 기록 (네이티브 윈도우 / 에뮬레이터)

### 3.1 사전 확인 사항

Claude Code 실행 환경: 네이티브 윈도우 (PowerShell/CMD/Git Bash)

Android SDK 설치 여부 확인을 위해 아래 명령으로 점검:

```
Test-Path "$env:LOCALAPPDATA\Android\Sdk"
echo $env:ANDROID_HOME
echo $env:ANDROID_SDK_ROOT
adb --version
sdkmanager --version
emulator -version
```

### 3.2 Android Studio SDK Components Setup

Android Studio 설치 마법사의 'SDK Components Setup' 화면에서 다음을 확인하고 진행:

- Android SDK Platform 체크 → 하위 Android 16.0 (Baklava, API 36.1) 선택
- SDK 위치: C:\Users\chois\AppData\Local\Android\Sdk (기본 경로 사용)
- Next → 라이선스 동의(Accept) → Finish 로 설치 진행

### 3.3 환경변수 설정 (ANDROID\_HOME, PATH)

SDK 폴더는 존재했으나 환경변수 미설정 상태였음 (adb 인식 안 됨). 아래 스크립트로 사용자 환경변수에 등록:

```
$sdkPath = "$env:LOCALAPPDATA\Android\Sdk"
```

```
[System.Environment]::SetEnvironmentVariable("ANDROID_HOME", $sdkPath, "User")
[System.Environment]::SetEnvironmentVariable("ANDROID_SDK_ROOT", $sdkPath, "User")

$paths = @(
    "$sdkPath\platform-tools",
    "$sdkPath\cmdline-tools\latest\bin",
    "$sdkPath\emulator"
)

$currentPath = [System.Environment]::GetEnvironmentVariable("Path", "User")
$newPath = $currentPath + ";" + ($paths -join ";")
[System.Environment]::SetEnvironmentVariable("Path", $newPath, "User")
```

적용 후 새 PowerShell 창에서 재확인 → adb, emulator는 정상 인식되었으나 sdkmanager는 미인식.

### 3.4 cmdline-tools 누락 문제 해결

원인: Android Studio 설치 시 'Command-line Tools'가 함께 설치되지 않아 cmdline-tools 폴더 자체가 없었음.

해결: Android Studio → More Actions/Settings → SDK Manager → SDK Tools 탭 → 'Android SDK Command-line Tools (latest)' 체크 → Apply로 설치.

설치 후 폴더 구조 확인 결과 cmdline-tools\latest\bin 아래 sdkmanager.bat, avdmanager.bat 등이 정상적으로 생성됨을 확인.

### 3.5 JDK 17 설치

sdkmanager 실행 시 'JAVA\_HOME is not set' 오류 발생. java 명령 자체도 PATH에 없었고, Android Studio 내장 JBR 경로도 존재하지 않아 별도 JDK 설치로 진행.

```
winget install EclipseAdoptium.Temurin.17.JDK
```

1차 시도 시 설치 관리자 종료 코드 1603(설치 실패) 발생 — 일반 권한 PowerShell에서 실행한 것이 원인으로 추정됨.

PowerShell을 관리자 권한으로 재실행 후 동일 명령 재시도 → 설치 성공.

새 PowerShell 창에서 확인:

```
java -version
# openjdk version "17.0.19" 2026-04-21

echo $env:JAVA_HOME
```

```
# C:\Program Files\Eclipse Adoptium\jdk-17.0.19-hotspot\
```

winget을 통한 MSI 설치가 JAVA\_HOME 및 PATH를 자동으로 설정함을 확인.

### 3.6 설치 완료 확인

아래 4개 명령이 모두 정상 작동함을 확인:

```
java -version      # JDK 17.0.19
adb --version      # Android Debug Bridge 1.0.41 (37.0.0)
sdkmanager --version # 21.0
emulator -version  # 36.6.11
```

### 3.7 시스템 이미지 및 AVD 생성

sdkmanager --list\_installed 결과, 이전 작업 흔적으로 다수의 플랫폼/빌드도구 및 system-images;android-33;google\_apis;x86\_64 이미지가 이미 설치되어 있음을 확인. 별도 다운로드 없이 바로 AVD 생성 진행.

```
avdmanager create avd -n Pixel6_API33 -k "system-images;android-33;google_apis;x86_64" -d pixel_6
```

생성 확인 결과 다음 두 개의 AVD가 존재함을 확인:

| AVD 이름                 | 디바이스              | API / ABI                                            |
|------------------------|-------------------|------------------------------------------------------|
| Pixel6_API33           | pixel_6 (Google)  | Android 13 Tiramisu / google_apis<br>x86_64          |
| Pixel_3a_API_33_x86_64 | pixel_3a (Google) | Android 13 Tiramisu / google_apis<br>x86_64 (기존 생성분) |

### 3.8 에뮬레이터 실행 및 연결 확인 (다음 단계)

```
emulator -avd Pixel6_API33

# 부팅 완료 후 새 창에서
adb devices
# emulator-5554  device ← 이렇게 뜨면 정상
```

세션 종료 시점 기준, 에뮬레이터 실행 및 adb devices 확인은 사용자 측에서 아직 진행 전 단계.

## 4. 다음 단계 (예정)

- emulator -avd Pixel6\_API33 실행 후 정상 부팅 및 adb devices 인식 확인
- 신규 안드로이드 프로젝트 생성 방식 결정 (빈 프로젝트 템플릿 vs 기존 구상안 기반)
- 프로젝트 루트에 CLAUDE.md 작성
- Claude Code 세션 시작 후 기능 단위 개발 루프(요청 → 빌드 → 테스트 → 로그 확인 → 커밋) 진행